



Aplicaciones Distribuidas

Semana 2 – Unidad 1:

Socket y Websocket

Grupo B:

Luna Mora Ernesto Gregory.
Aucatoma Celorio Jhinson Stalyn.
Pacheco Cardenas Cristhian Daniel.
Sanchez Pilaloo Andy Paul.



Introducción



- Las aplicaciones modernas necesitan comunicarse entre sí a través de una red
- Para eso existen protocolos y mecanismos de comunicación como Sockets, TCP y UDP
- Estos conceptos son la base de cualquier sistema que intercambie datos entre máquinas

Sistema Distribuido

- Es un conjunto de procesos o computadoras que se comunican entre sí a través de una red
- Cada parte puede estar en una máquina diferente, pero trabajan juntas como un solo sistema
- Se basan en el modelo cliente-servidor, el cliente solicita y el servidor responde
- La comunicación entre ellos se logra mediante protocolos de red como TCP y UDP



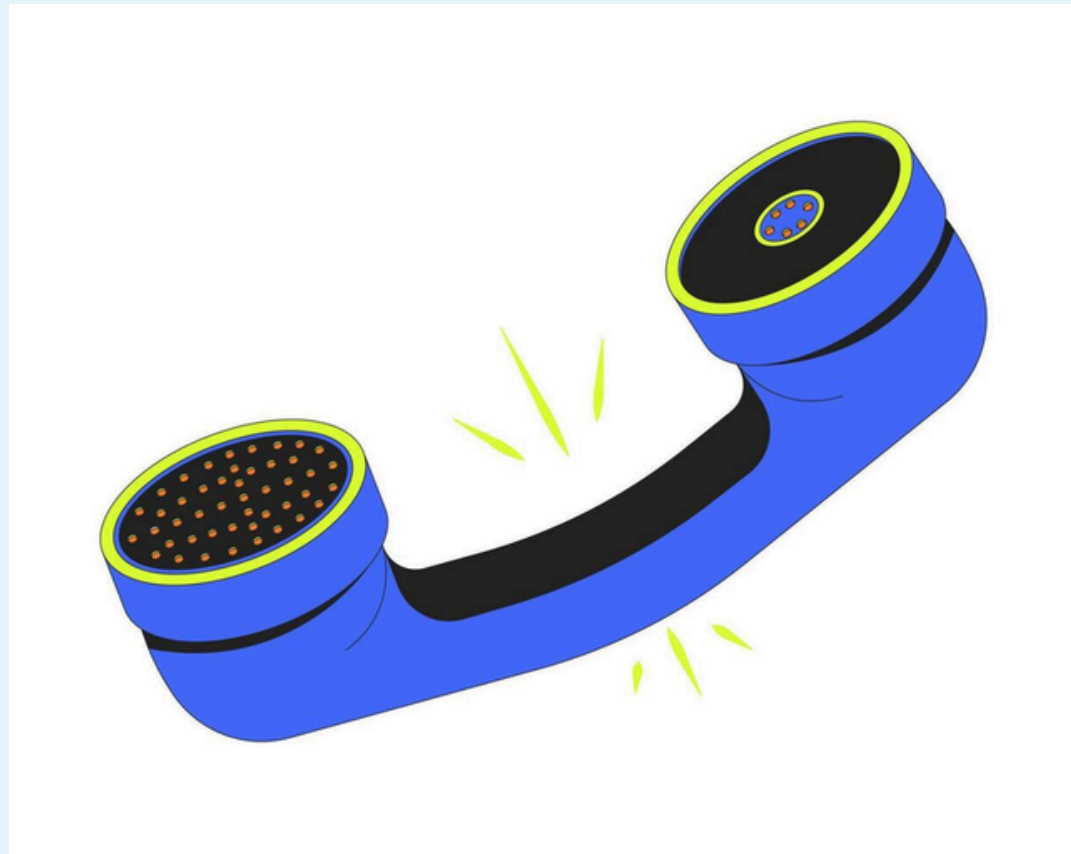
TCP vs UDP

- **TCP:** Protocolo orientado a conexión, establece un canal antes de enviar datos, garantiza entrega y orden, más lento. Se usa cuando la fiabilidad es prioritaria, como en transferencia de archivos o sistemas bancarios
- **UDP:** Protocolo sin conexión, envía los datos directamente sin verificación, más rápido y liviano. Se usa cuando la velocidad es prioritaria, como en videollamadas o streaming



¿Qué es un Socket?

- Es el punto de conexión entre dos procesos que se comunican por red
- Funciona como un canal bidireccional de entrada y salida de datos entre cliente y servidor
- Puede operar sobre TCP, garantizando fiabilidad, o sobre UDP, priorizando velocidad



Tipos de Socket



Transmission Control Protocol

- Orientado a conexión.
- Comunicación confiable.
- Garantiza orden de datos.
- Usa control de errores.

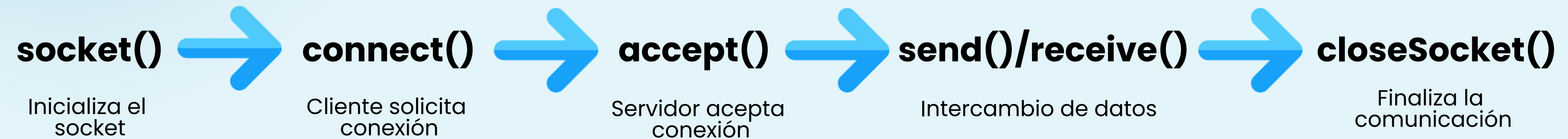


User Datagram Protocol

- Comunicación sin conexión.
- Mayor velocidad.
- Menor confiabilidad.
- Permite multidifusión.



Ciclo de vida de un Socket TCP



¿Qué es WebSocket?

Protocolo de comunicación bidireccional y full-duplex sobre una conexión TCP persistente.

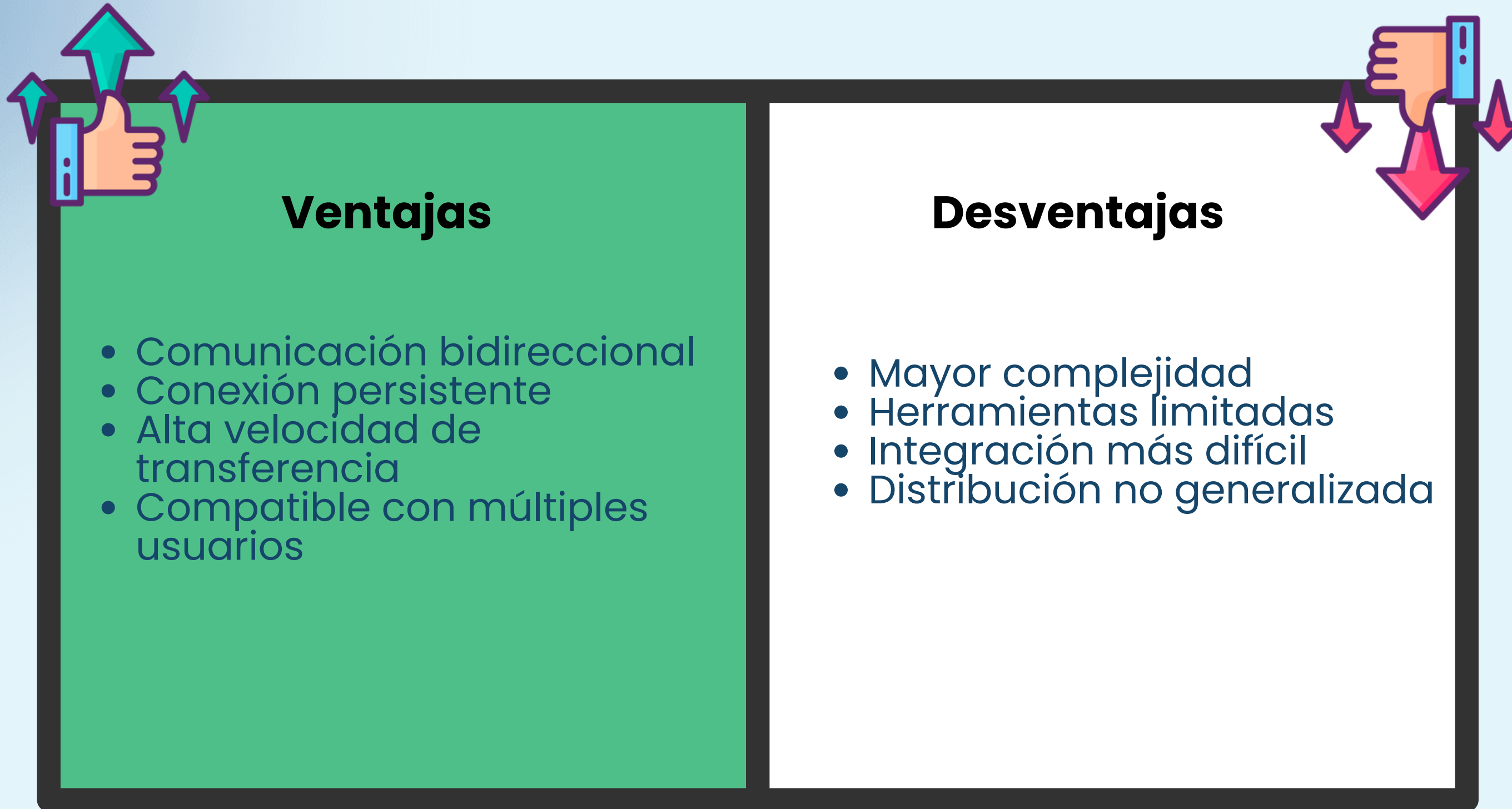
WebSocket Connection



Tiempo real - Full Duplex - Persistencia



Ventajas y Desventajas de WebSocket





Sockets vs WebSockets



Sockets

Comunicación directa entre aplicaciones usando TCP o UDP.
Son comunes en servidores, bases de datos, videojuegos y sistemas internos.



WebSockets

Comunicación persistente entre navegador y servidor.
Ideal para aplicaciones web en tiempo real como chats, notificaciones y dashboards



Diferencia clave

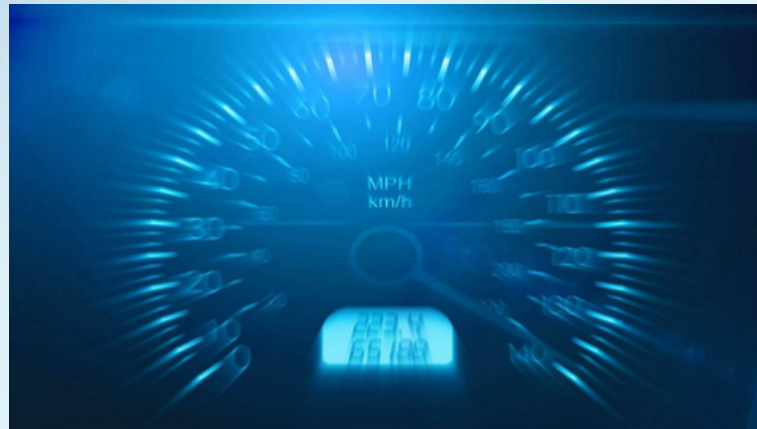
Sockets = más control técnico
WebSockets = mejor integración web



Latencia y Rendimiento

Sockets TCP

Baja latencia en redes locales y comunicación confiable



Sockets UDP

Mayor velocidad, pero sin garantía de entrega



WebSocket

Buen rendimiento en aplicaciones web porque mantiene la conexión abierta





SEGURIDAD

Sockets



Pueden protegerse con TLS en TCP o DTLS en UDP

WebSocket



Debe usarse como WSS, es decir, WebSocket seguro sobre TLS

Punto importante



WebSocket no trae autenticación propia. Se debe implementar con JWT, sesiones, OAuth o roles



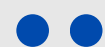
ESCALABILIDAD Y COMPATIBILIDAD

Compatibilidad



WebSocket funciona en navegadores modernos y usa puertos comunes como 80 y 443.

Escalabilidad



Puede manejar muchas conexiones simultáneas con arquitecturas orientadas a eventos.



Sockets



Escalan bien en redes privadas, pero requieren mayor configuración manual

Idea principal



WebSocket facilita la web; Sockets destacan en sistemas internos



Casos de Uso: Sockets

VoIP & Video



Uso de UDP Sockets para llamadas de voz y video donde la velocidad prima sobre la integridad perfecta.

Juegos Nativos

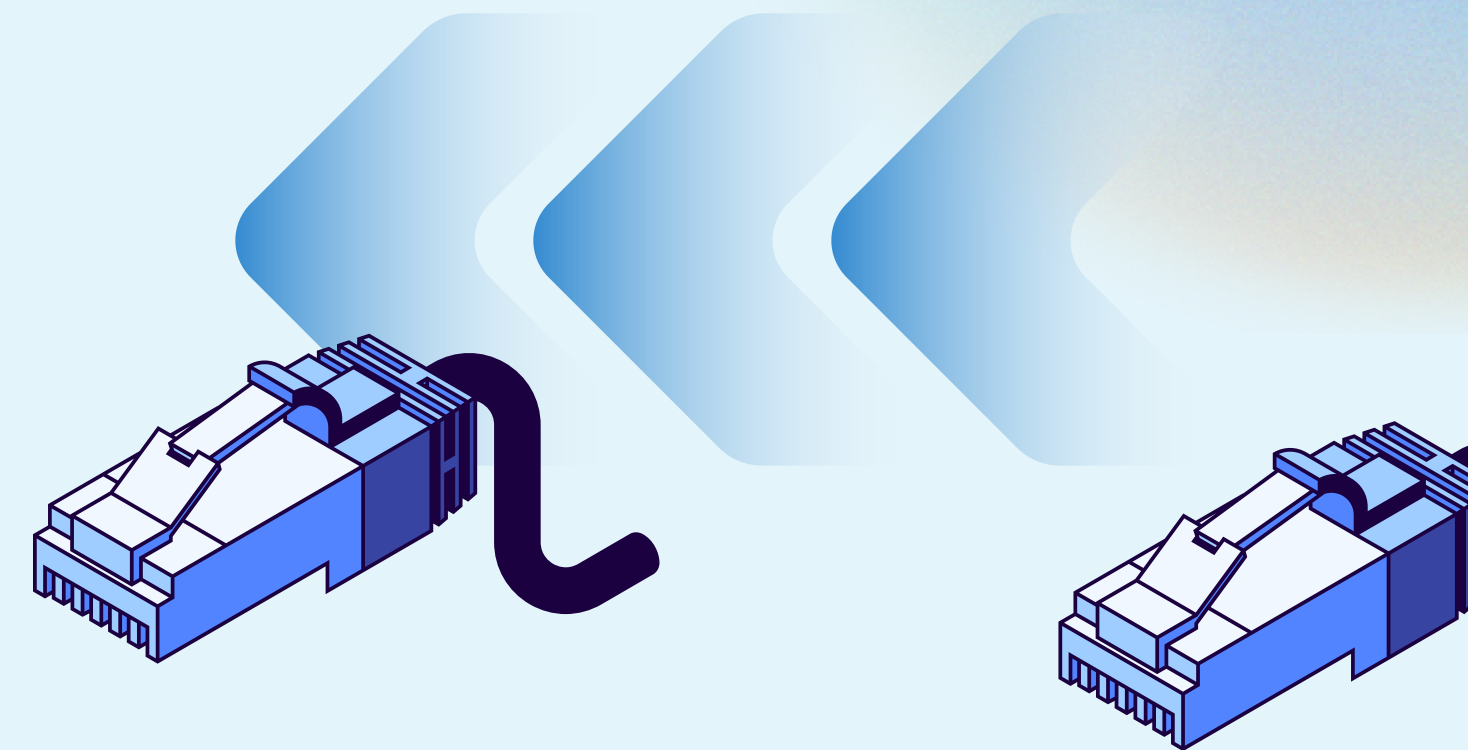


Comunicación de baja latencia en juegos de disparos o acción (Fast-paced games) usando protocolos binarios.

Servicios Core



Protocolos como FTP, SSH o SMTP que requieren una conexión persistente y fiable a nivel de sistema operativo.



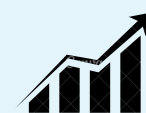


Casos de Uso: WebSockets



Chat Apps

Mensajería instantánea en navegadores (WhatsApp Web, Discord) sin necesidad de refrescar la página.



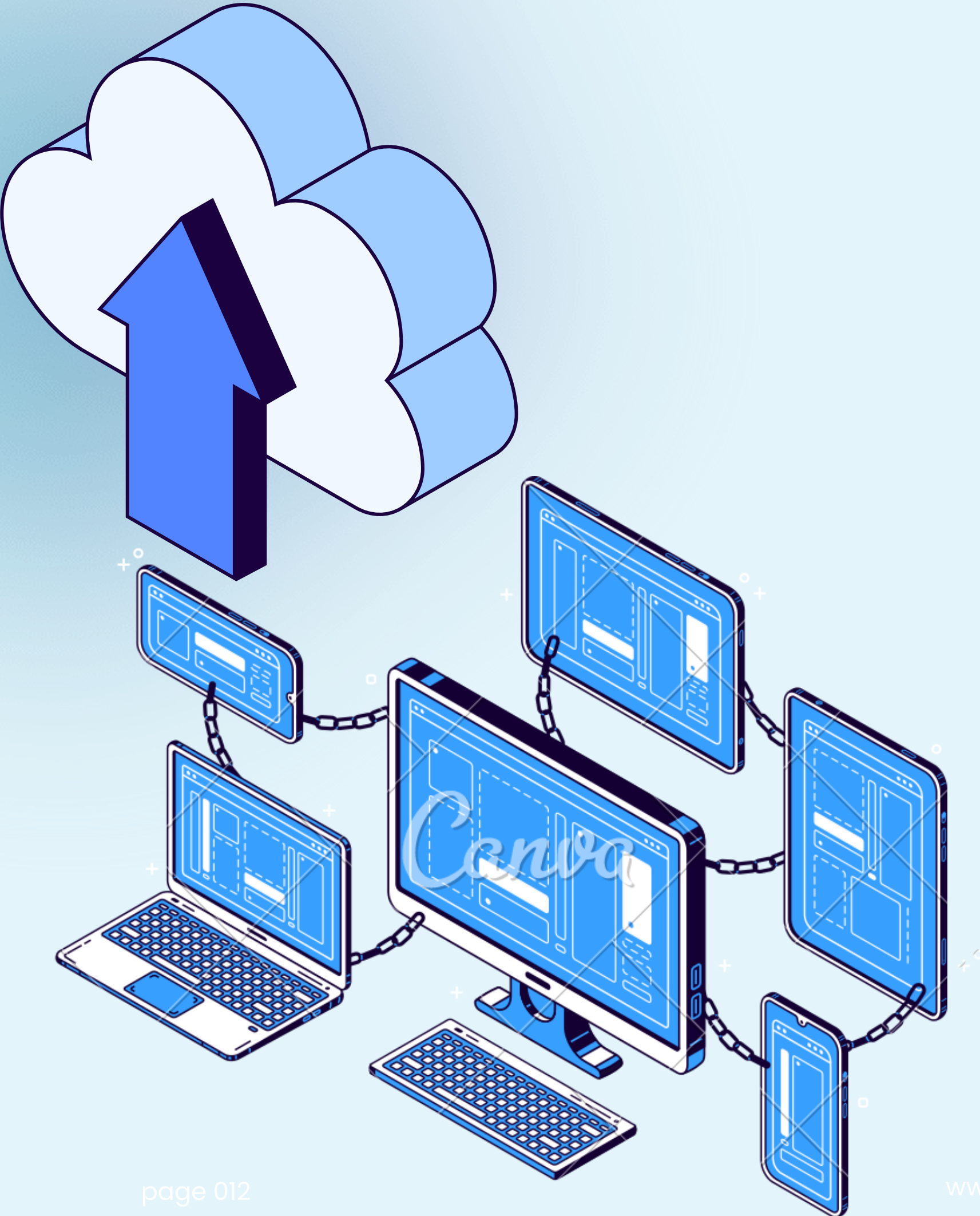
Trading Dashboards

Visualización de precios de acciones o criptomonedas en tiempo real con actualizaciones constantes.



Colaboración

Edición simultánea de documentos (Google Docs) o pizarras blancas digitales en vivo.



Conclusiones

- **Elección:** Use Sockets para bajo nivel/apps nativas y WebSockets para la Web moderna.
- **Rendimiento:** Ambos superan al modelo HTTP tradicional en escenarios de tiempo real.
- **Futuro:** Con la llegada de HTTP/3 y WebTransport, el ecosistema seguirá evolucionando.

